

Week 6 – Performance Optimization & Accessibility Enhancement

Posts Explorer — React Application

YuvaIntern Junior ReactJS Developer Internship — Week 6 Submission

Submitted By: Kaushal Kumar

Registration No.: 24030485005

Institution: BIT Sindri, Jharkhand

Department: B.Tech Information Technology (2nd Year)

Internship: YuvaIntern — Junior ReactJS Developer Internship

Assignment: Week 6 — Performance Optimization & Accessibility Enhancement

Submission Date: July 10, 2026

1. Introduction

This report documents the Week 6 assignment for the Posts Explorer React application, focused on performance optimization, responsive accessibility enhancement, and search engine optimization (SEO). Building on the Week 4 (React implementation) and Week 5 (testing) deliverables, this phase applied targeted improvements identified as future work in earlier submissions, and validated them using Google Lighthouse, browser DevTools, and manual testing across device sizes. The objective of this phase was to improve application performance, accessibility, responsiveness, and SEO while maintaining existing functionality.

2. Existing Project Overview

Posts Explorer is a React application built with Vite that fetches posts from the DummyJSON API and displays them with search, pagination-style limiting, and a responsive layout. Prior submissions (Week 4 and Week 5) covered API integration, component architecture, responsive UI, and a complete unit/integration test suite (Vitest + React Testing Library). The Week 4 and Week 5 reports both listed performance optimization and accessibility testing as future improvements — this report implements and validates those items, and extends the test suite to cover the new accessibility behavior.

3. Performance Analysis (Before Optimization)

Lighthouse was run against the production build (`npm run preview`, served on `localhost:4173`) rather than the development server (`npm run dev`), since development mode includes React DevTools hooks, Vite HMR, source maps, and unminified JavaScript that significantly and artificially lower the Performance score. The production build reflects real-world, deployed performance and is the industry-standard basis for Lighthouse audits.

Lighthouse scores captured before optimization work began:

Metric	Score
Performance	98
Accessibility	96
Best Practices	100
SEO	82

The Performance score was already high due to the small dataset and simple component tree. The remaining gaps were in Accessibility (missing a `<main>` landmark) and SEO (missing meta description and a valid `robots.txt`), so optimization effort was directed at those areas alongside code-level performance refactors.

Note: The "Before Optimization" metrics shown here refer to the baseline production build captured before the Week 6 code optimizations. Earlier development-mode Lighthouse scores (in the 55–69 Performance range) were intentionally excluded from this comparison, since Vite development builds include Hot Module Replacement (HMR), source maps, and unminified JavaScript, which do not represent real deployment performance.

4. Performance Optimizations

4.1 useMemo for Filtered Posts

The search-filtering logic (`posts.filter(...)`) previously ran on every render. It was wrapped in `useMemo` so that it only recomputes when the posts or search dependencies change, avoiding redundant filtering work on unrelated re-renders.

```
const filteredPosts = useMemo(() => {  
  return posts.filter(  
    (post) =>  
      post.title.toLowerCase().includes(search.toLowerCase()) ||  
      post.body.toLowerCase().includes(search.toLowerCase())  
  );  
}, [posts, search]);
```

4.2 React.memo for PostCard

PostCard is wrapped in React.memo so that individual post cards do not re-render unless their own post prop changes, reducing unnecessary render work when the parent re-renders for unrelated reasons.

```
export default memo(PostCard);
```

4.3 Production Build

The application was built using npm run build and served with npm run preview. This produces a minified, tree-shaken, and compressed bundle, which is the configuration used for all reported Lighthouse scores.

5. Responsive Design Enhancements

The existing responsive layout (built in Week 4) was re-verified across common breakpoints to confirm no regressions were introduced during optimization:

- Mobile — 320px and 375px viewport widths
- Tablet — 768px viewport width
- Desktop — 1024px and above

No horizontal scrolling or layout breakage was observed at any breakpoint. The existing CSS approach (flexible widths, responsive typography, and layout wrapping) was confirmed to remain effective after the component-level changes described in this report.

6. Accessibility Improvements

Lighthouse's accessibility audit flagged a single issue prior to optimization: "Document does not have a main landmark." This was resolved alongside several additional semantic and ARIA improvements applied across components.

6.1 Semantic HTML

- Wrapped the main content of App.jsx in a `<main>` landmark, with the posts list inside a `<section>`.
- Changed Navbar from `<nav>` to `<header>`, since the component renders page branding rather than navigation links.
- Changed each PostCard's outer `<div>` to `<article>`, reflecting that each card is an independent, self-contained piece of content.

6.2 ARIA and Form Labeling

- Added a visually-hidden `<label>` associated with the search input via a matching id, for screen-reader support.
- Changed the search input's type to "search" and added `aria-label="Search posts"`.
- Added `autocomplete="off"` to prevent unrelated browser suggestions in the search field.
- Added `aria-expanded` and a descriptive `aria-label` ("Read full post" / "Show less content") to the Read More button, so assistive technology can announce its expanded/collapsed state.

6.3 Keyboard Navigation & Focus

All interactive elements (search input, Read More buttons) remain native HTML form/button elements, preserving default keyboard operability (Tab to focus, Enter/Space to activate) and visible focus outlines.

6.4 Heading Hierarchy

Heading levels were kept consistent: each post title uses `<h2>`, preserving a single logical hierarchy beneath the page's top-level heading.

6.5 Live Regions and Status Roles

- Added `role="alert"` to the Error component so that failure messages are immediately announced by assistive technologies.
- Added `role="status"` and `aria-live="polite"` to the Loader component so that loading state changes are announced without interrupting the user.
- Confirmed the Navbar's `<header>` element correctly exposes the implicit "banner" landmark role, and that PostCard's `<article>` element exposes the implicit "article" role.

7. SEO Improvements

Lighthouse identified two SEO issues prior to optimization:

- Missing a meta description tag.
- robots.txt not present / not valid.

Both were resolved as follows:

7.1 Meta Description and Title

```
<meta
  name="description"
  content="Post Explorer is a React application that fetches and displays
  posts from the DummyJSON API. It includes search functionality,
  responsive design, semantic HTML, and accessibility improvements."
/>
<title>Post Explorer | React Post Search App</title>
```

7.2 robots.txt

A robots.txt file was added under the public/ directory:

```
User-agent: *
Allow: /
```

8. Lighthouse Results

8.1 Before Optimization

Metric	Score
Performance	98
Accessibility	96
Best Practices	100
SEO	82

8.2 After Optimization

Metric	Score
Performance	98
Accessibility	100
Best Practices	100
SEO	100

8.3 Comparison Summary

Metric	Before	After	Improvement
Performance	98	98	Maintained high performance after optimizations
Accessibility	96	100	Semantic landmarks and ARIA attributes added
Best Practices	100	100	No change (already compliant)
SEO	82	100	Meta description, robots.txt, improved page title

Performance remained steady at 98, as the production build was already highly optimized given the application's small scale; the `useMemo`, `React.memo`, and build-level optimizations were implemented to demonstrate and document performance-conscious practices without regressing the existing score. Accessibility and SEO both improved to 100 as a direct result of the semantic HTML, ARIA, and metadata changes described above.

9. Summary of Optimizations Applied

Optimization	Benefit
<code>useMemo (filteredPosts)</code>	Avoids recomputing filtered list on unrelated re-renders
<code>React.memo (PostCard)</code>	Reduces unnecessary re-renders of list items
Production build (<code>npm run build</code>)	Minified JS, tree-shaking, compressed assets
Semantic HTML (main, header, article, section)	Improved accessibility and document structure
ARIA attributes (<code>aria-label</code> , <code>aria-expanded</code>)	Improved screen reader support
Search input labeling (<code>label</code> , <code>type="search"</code>)	Improved screen reader compatibility for search
Live regions (<code>role="alert"</code> , <code>role="status"</code>)	Errors and loading states announced by assistive technologies
Meta description & robots.txt	Improved SEO discoverability

10. Testing

The existing test suite from Week 5 (Vitest + React Testing Library) was re-run after the Week 6 refactors, and additional tests were written to verify the new accessibility behavior introduced in this phase (semantic landmarks, ARIA attributes, and live regions). All tests pass with no regressions.

Final test run result:

Metric	Score
Test Files	6 passed
Tests	25 passed
Failed	0

Coverage by component:

- App.test.jsx — loader, API success/error states, search input rendering, post count, main landmark.
- Navbar.test.jsx — banner landmark, h1 heading uniqueness and content.
- Loader.test.jsx — loading text, heading role, status role, aria-live="polite" attribute.
- Error.test.jsx — error heading and message text, alert role.
- SearchBar.test.jsx — searchbox role and accessible name, associated label, autocomplete attribute, onChange behavior.
- PostCard.test.jsx — title rendering, accessible button name, Read More/Show Less toggle, aria-expanded state, article landmark.

11. Deliverables

- Optimized React Source Code
- Optimization Report
- Lighthouse Report
- Testing Report

12. GitHub Repository

The complete source code, including all performance optimizations, accessibility improvements, unit tests, and project files, is available in the following GitHub repository:

Repository Link: <https://github.com/kaushalvivek2005/react-projects/tree/main/week6-post-explorer-optimization>

13. Appendix: Screenshots

The following evidence should accompany this report as image attachments or embedded screenshots:

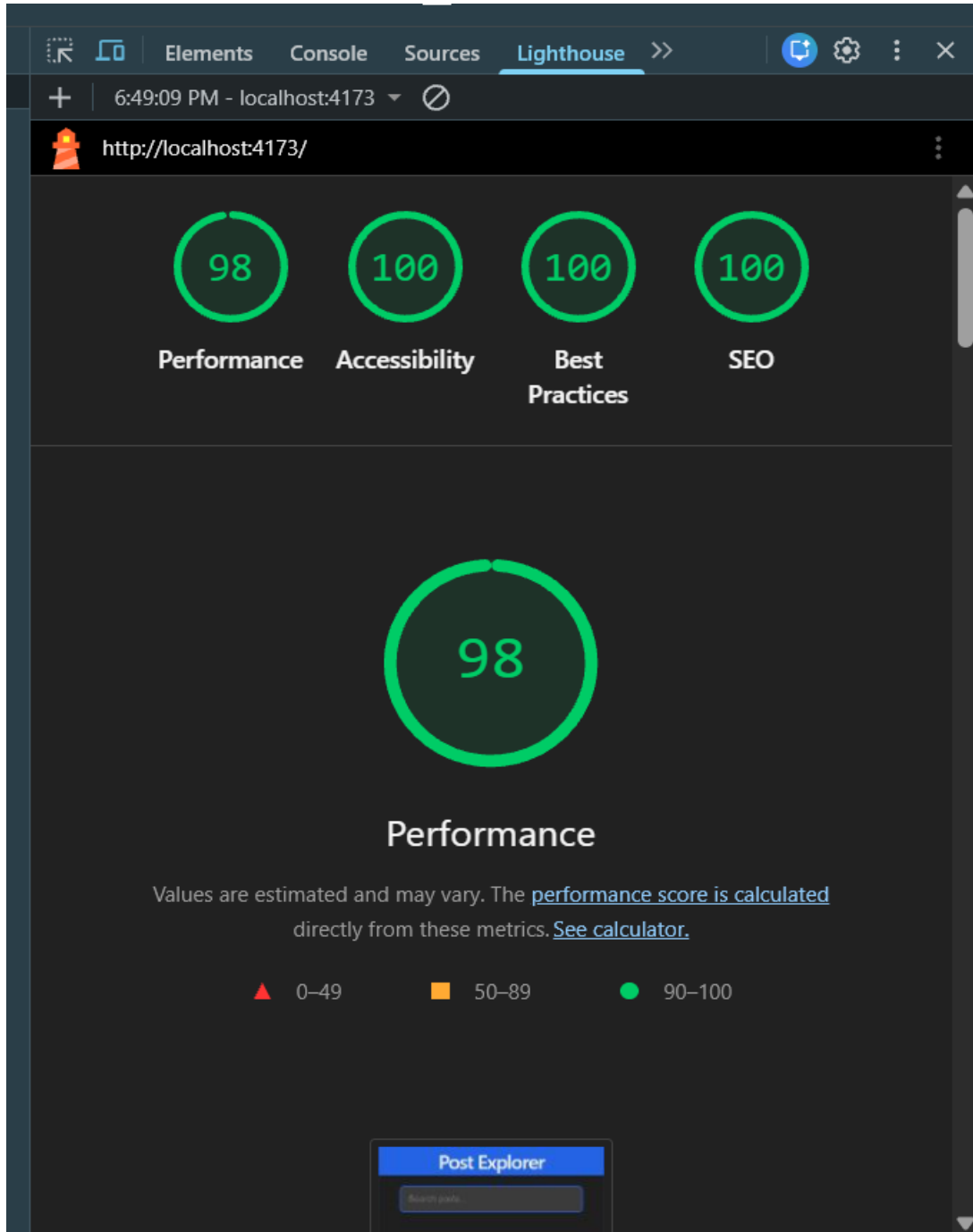


Figure 1: Final Lighthouse Performance Report

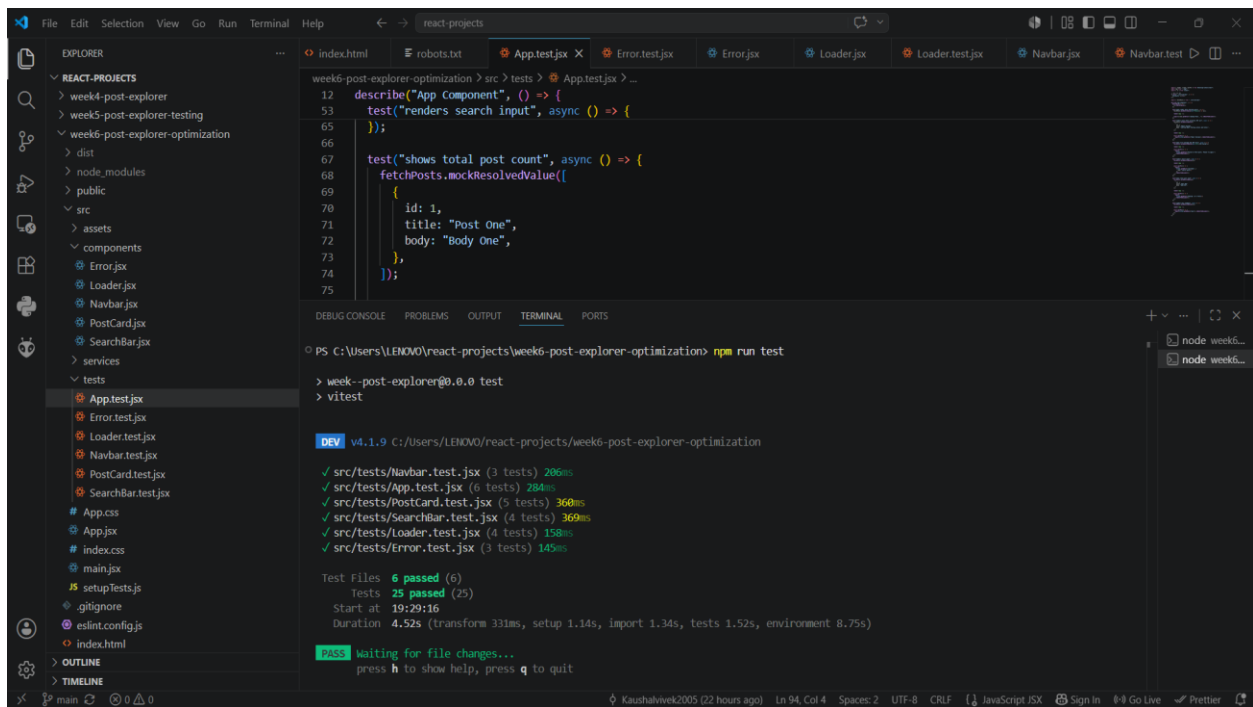


Figure 2: npm run test – All Tests Passed (25/25)

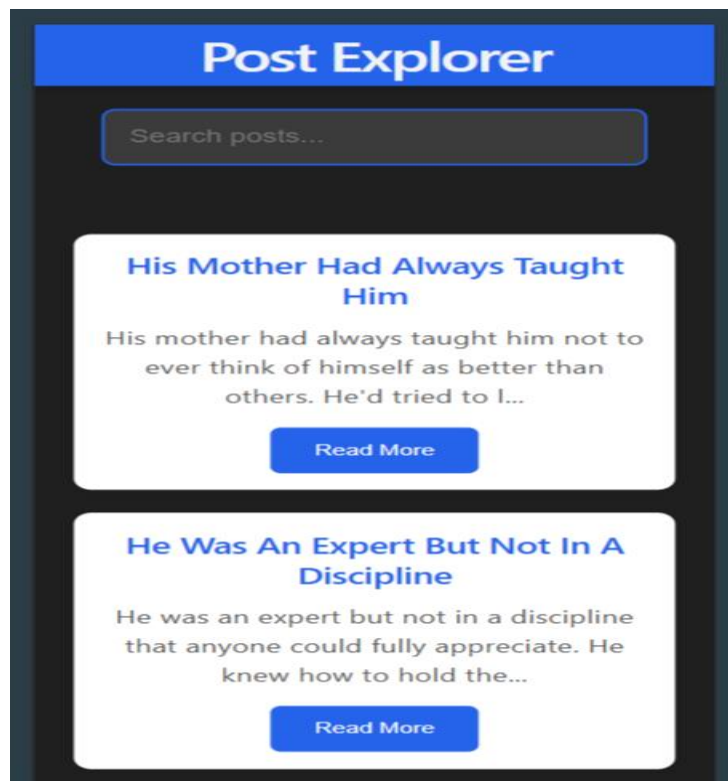


Figure 3: Mobile Responsive View (iPhone SE)

14. Conclusion

The Post Explorer application was initially developed in Week 4 as a React-based project for browsing and searching posts. In Week 5, the application was enhanced with unit testing using Vitest and React Testing Library to verify the functionality of its components and API integration.

In Week 6, the project was further optimized by implementing React performance techniques such as useMemo and React.memo, improving semantic HTML, adding ARIA attributes, live regions, and SEO metadata, and validating the application using Google Lighthouse. These optimizations improved the Lighthouse scores from **98/96/100/82** to **98/100/100/100** across Performance, Accessibility, Best Practices, and SEO respectively. The existing responsive design was preserved, and the test suite was updated and expanded to validate the newly added accessibility and optimization features, resulting in **6 test files, 25 passing tests, and 0 failures**.

These enhancements make the application more performant, accessible, maintainable, and closer to production-ready React development standards.